ESCUELA
COLOMBIANA
DE INGENIERÍA
JULIO GARAVITO

Excepciones

Laboratorio 4

Oscar Lasso

Juan Diego Gaitán

Profesor:

Maria Irma Diaz Rozo

Juan Frasica

Desarrollo Orientado a Programación de Objetos

Escuela Colombiana de Ingeniería Julio Garavito

Bogotá D.C.

ESCUELA COLOMBIANA DE INGENIERÍA
DESARROLLO ORIENTADO POR OBJETOS
Excepciones
2026-1 — Laboratorio 4/6

OBJETIVO

Evidenciar el logro de los siguientes resultados de aprendizaje:

1. Perfeccionar el diseño y código de un proyecto considerando casos especiales y errores.
2. Construir clases de excepción encapsulando mensajes.
3. Manejar excepciones considerando los diferentes tipos.
4. Registrar la información de errores que debe conocer el equipo de desarrollo de una aplicación en producción.
5. Practicar las prácticas

   Designing. Simplicity.

   Designing. Refactor whenever and wherever possible.

CONTEXTO

Reto: Desarrollar un observatorio digital de las selecciones que participan en la **Copa Mundial de la FIFA 2026**

La **Copa Mundial de la FIFA 2026** será la vigésima tercera edición del torneo, pero será la primera en contar con 48 equipos y tres países anfitriones. En esta edición se contará con un nuevo indicador: el **VMPE**. [FIFA]. El **VMPE** (Valor de Mercado Ponderado de Equipo) es un indicador desarrollado para estimar la calidad de los equipos valorando el precio de mercado de sus jugadores y el porcentaje de participación en los partidos de la selección. El **precio de mercado** se establece teniendo en cuenta su edad, su rendimiento, su potencial, su contrato actual y la demanda de los clubes. El **porcentaje de participación** se calcula considerando los minutos jugados por el jugador y los minutos jugados por todos los jugadores de la selección.	2026 FIFA WORLD CUP UNITED STATES CANADA MÉXICO

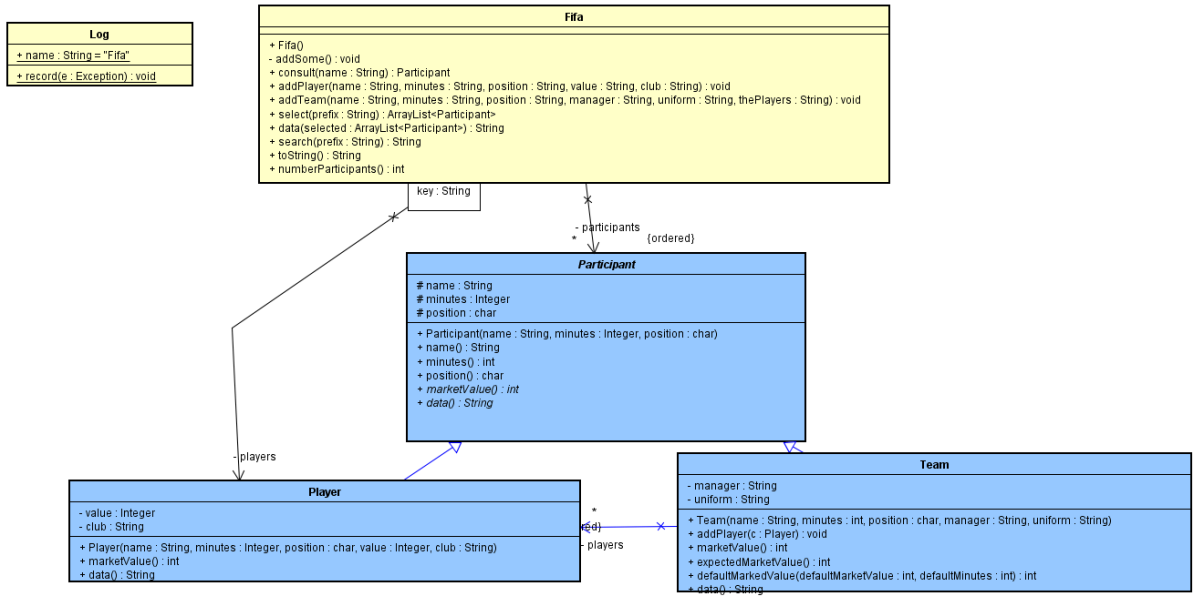
PARTE I. Practicando con excepciones

A MDD y BDD con `fifa2026` [En lab04.doc fifa2026.astah *.java]

En este punto vamos a aprender a diseñar, codificar y probar usando excepciones. Para esto se van a trabajar algunos métodos de la clase `Team`.

1. En su directorio descarguen los archivos contenidos en `team.zip`, revisen el contenido y estudien el diseño estructural de la aplicación (únicamente la zona en azul).
2. Expliquen por qué el proyecto no compila. Realicen las adiciones necesarias para lograrlo.
3. Dado el diseño y las pruebas, documenten (si es necesario) y codifiquen el método `marketValue`.
4. Dada la documentación y el diseño, codifiquen y prueben el método `expectedMarkedValue`.
5. Documenten, diseñen, codifiquen y prueben el método `helpedMarkedValue`.
6. Propongan el cálculo, documenten, diseñen, codifiquen y prueben el método `bestMarkedValue`.

1.



2. No compila debido a que no existe la clase FifaException en el paquete. Primero se debe implementar

```java
public class Team extends Participant{

    private String manager;
    private String uniform;

    private ArrayList<Player> players;

    /**
     * Constructs a new Team
     * @param name
     * @param type
     */
    public Team(String name, int minutes, char position, String manager, String uniform){
        super(name, minutes, position);
        this.manager=manager;
        this.uniform=uniform;
        players= new ArrayList<Player>();
    }


     /**
     * Add a new Player
     * @param c
     */
    public void addPlayer(Player c){
        players.add(c);
    }
```

-Ahora si se implementa:

```java
package domain;


/**
 * Clase de excepciones para Fifa
 *
 * @author Juan Diego Gaitan, David Lasso
 * @version 1.0
 */
public class FifaException extends Exception
{

    public static final String VALUE_UNKNOWN = "Marker value is unknown";
    public static final String IMPOSSIBLE = "The resistance cannot be calculate";
    public static final String MINUTES_UNKNOWN = "Minutes is unknown";

    public FifaException(String message)
    {
        super(message);

    }

}
```

IMPLEMENTACIÓN DE MARKET VALUE

IMPLEMENTACIÓN DE EXPECTED VALUE

IMPLEMENTACIÓN DEFAULT VALUE

IMPLEMENTACIÓN Y PROPUESTA BEST MARKET VALUE

PARTE II. Desarrollando `fifa2026`

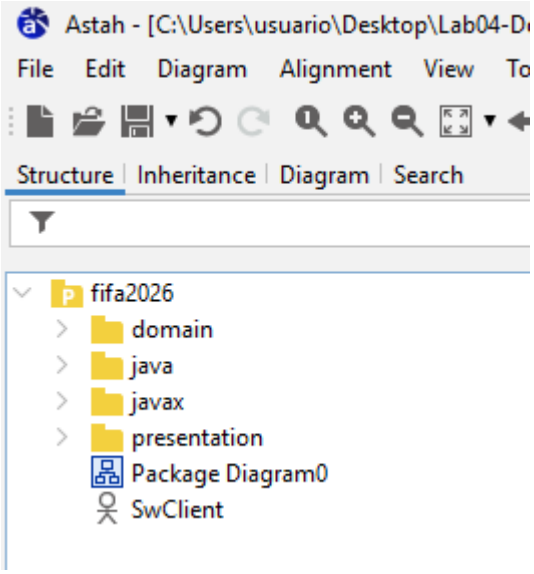
A Revisando línea base [En lab04.doc]

1. En su directorio descarguen los archivos contenidos en `fifa2026Others.zip`, revisen el contenido. (a) ¿Cuántos archivos se tienen? (b) ¿Cómo están organizados? (c) ¿Cómo deberían estar organizados?
2. Estudien el diseño del programa: diagramas de paquetes y de clases. (a) ¿Cuántos paquetes tenemos? (b) ¿Cuántas clases tiene el sistema? (c) ¿Cómo están organizadas? (d) ¿Cuál es la clase ejecutiva?
3. Incluya los nuevos archivos en los directorios correspondientes para ejecutar el proyecto, (a) ¿Qué estructura debe tener? (b) ¿Qué clases deben tener cada directorio? (c) ¿Cómo los incluyeron?
4. Ejecute el proyecto, (a) ¿Qué funcionalidades ofrece? (b) ¿Cuáles funcionan?
5. Revisen el código y la documentación del proyecto. (a) ¿De dónde salen los datos iniciales? (b) ¿Qué clase pide que se adicionen? (c) ¿Qué clase los adiciona?

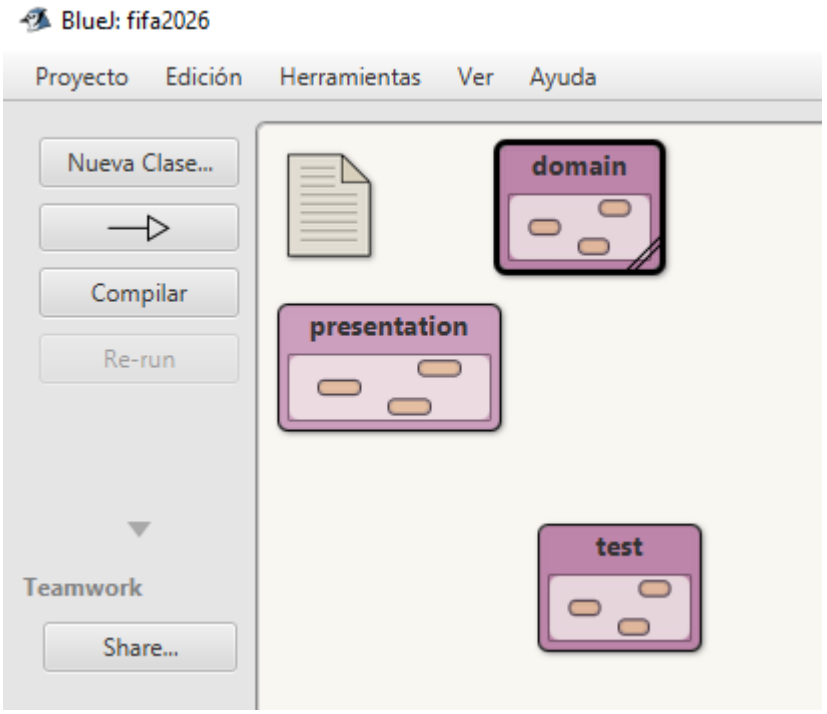
equipo › Escritorio › fifa2026Others

Nombre	Fecha de modificación	Tipo	Tamaño
Fifa	25/03/2026 9:28 p. m.	Archivo de origen ...	4 KB
FifaGUI	25/03/2026 10:47 p. m.	Archivo de origen ...	8 KB
Log	25/03/2026 9:28 p. m.	Archivo de origen ...	1 KB

1. a) Tiene 3 archivos de clase .Java
   b) Están sueltos y sin ningún tipo de organización de paquete o jerarquía
   c) Deberían estar organizados en la carpeta principal FIFA y en sus respectivos paquetes.



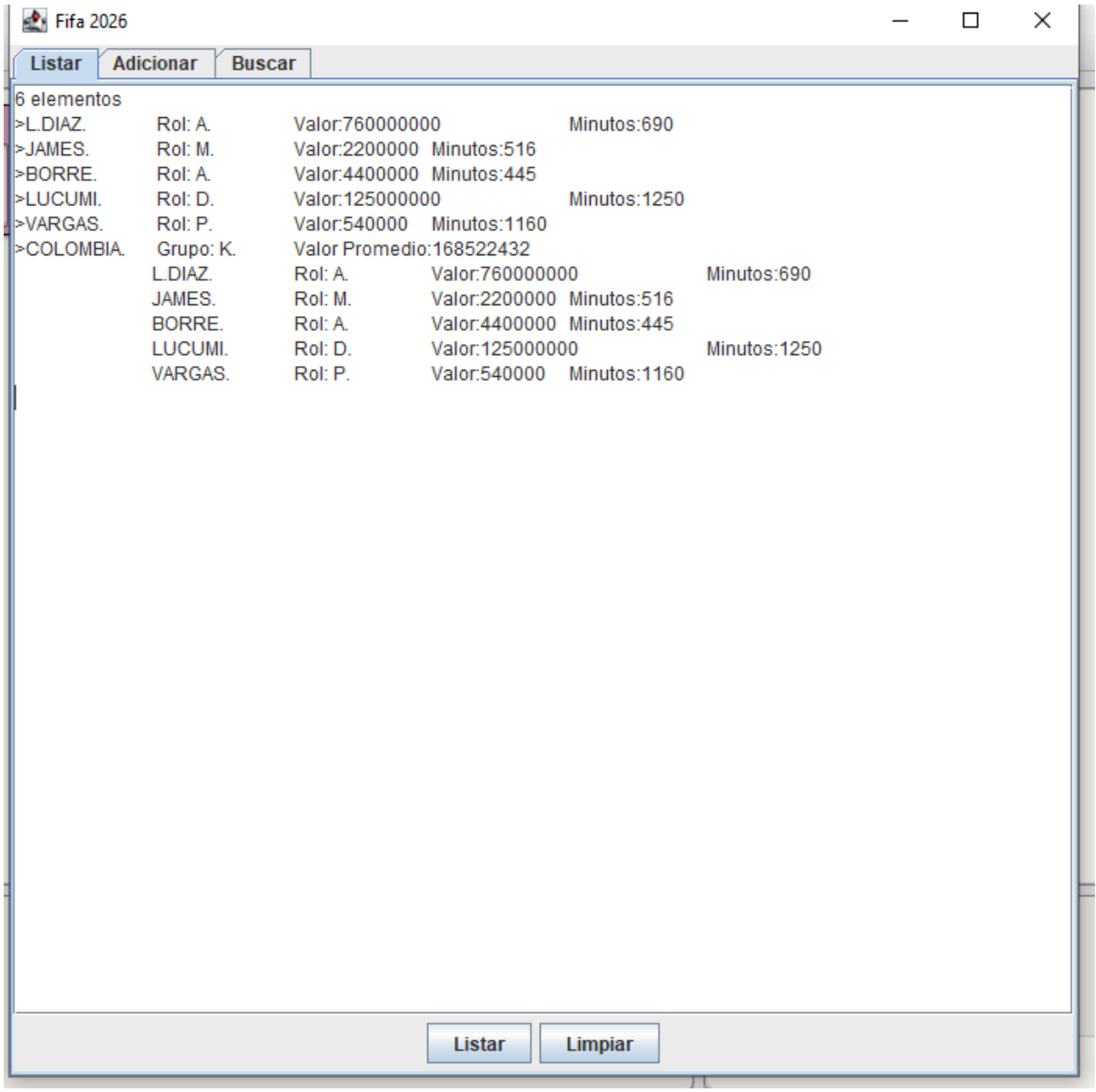
2. a) En este momento existen 4 paquetes en el diagrama de paquetes: domain, presentation, java y javax
   b)El sistema tiene 8 clases en total.
   c) Están organizadas por paquetes con herencias, clases de test, clases puras.
   d) La clase ejecuta es la presentation pues esta es donde se ejecuta el main() para poder visualizar la totalidad del sistema.



3. a) FIFA y LOG deben estar en domain. y FIFA GUI debe estar en presentation
   b) Domain finalmente queda con Fifa, Participant, Team, Player y Log. Presentation con FIFAGUI

y Test con TeamTest

c) En Bluej se deben añadir manualmente el archivo .Java a cada paquete.



4. a)Se abre un panel con dos botones en la zona inferior; “Listar” para mostrar información sobre jugadores, estadísticas y equipos, “Limpiar” para borrar en pantalla la información. Además en la zona superior hay diferentes pestañas como Listas, Adicionar y Buscar.
b) Listar muestra correctamente la información. Limpiar limpia correctamente. Adicionar se puede llenar el formulario pero no se adiciona correctamente. Buscar no funciona.

5. a) Los datos iniciales salen de la clase Fifa en domain. Cuando se llama al constructor de Fifa se llama a un método privado addSome() que adiciona los datos iniciales de ejemplo.
   b) En FifaGUI, en el constructor se instancia un nuevo Fifa y ahí se llama al método addSome() de Fifa.
   c) Los adiciona Fifa en domain, pero se llama addPlayer() y addTeam() internamente para añadir jugadores y equipos.

```java
/**
 * Create a Fifa
 */
public Fifa(){
    participants = new ArrayList<Participant>();
    players = new TreeMap<String,Player>();
    addSome();
}

private void addSome(){
    String [][] players= {{"L.DIAZ", "690","A","760000000","Bayer"},
                          {"JAMES", "516","M","2200000","Minnesota"},
                          {"BORRE", "445","A","4400000","Sport Club"},
                          {"LUCUMI", "1250","D","125000000","Bologna"},
                          {"VARGAS", "1160","P","540000","Atlas"}};
    for (String [] p: players){
        addPlayer(p[0],p[1],p[2],p[3],p[4]);
    }

    String [][] teams = {{"COLOMBIA","1620", "K", "Lorenzo", "Amarill-Rojo-Azul", "L.DIAZ\nJAMES\nBORRE\nLUCUMI\nVARGAS"}};
    for (String [] t: teams){
        addTeam(t[0],t[1],t[2],t[3],t[4],t[5]);
    }
}
```

B Realizando inversa. Adicionar y listar. Todo OK. [En lab04.doc fifa2026.astah *.java]

(NO OLVIDE BDD – MDD)

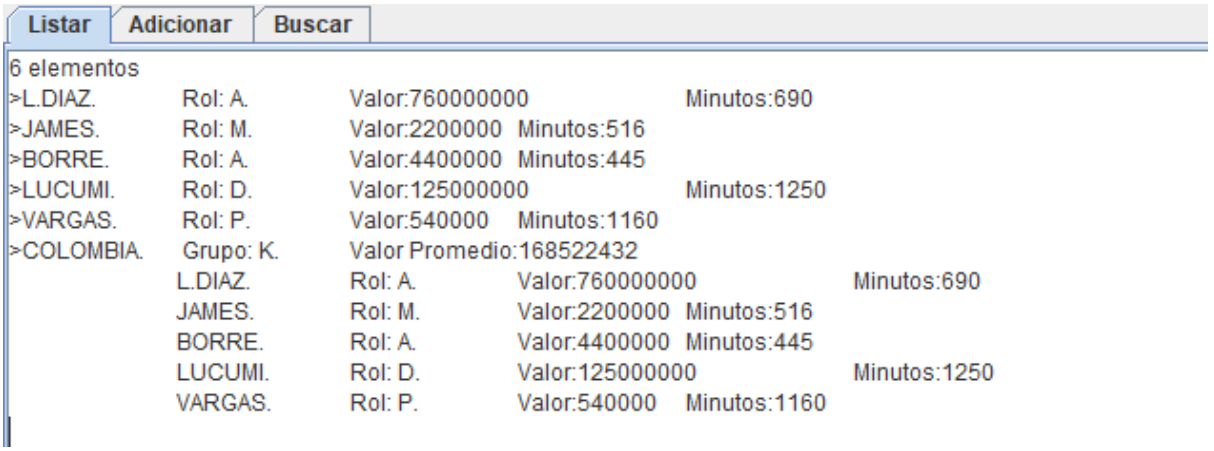
El objetivo es realizar ingeniería inversa a las funciones de adicionar y listar.

1. Adicionen un nuevo equipo y un nuevo jugador. `Jugador` MESSI, D, 1420, Inter, 15000000 `Equipo`
   Argentina, J, Scaloni, 1620, Azul-blanco (a) ¿Qué ocurre? (b) ¿Cómo lo comprueban? Capturen la pantalla. (c) ¿Es adecuado este comportamiento? Revisen el código asociado a adicionar en la capa de presentación y la capa de dominio. (a) ¿Qué método es responsable en la capa de presentación? (b) ¿Qué método en la capa de dominio?
2. Realicen ingeniería inversa para la capa de dominio para adicionar. Capturen los resultados de las pruebas de unidad.
3. Revisen el código asociado a listar en la capa de presentación y la capa de dominio. (a) ¿Qué método es responsable en la capa de presentación? (b) ¿Qué método en la capa de dominio?
4. Realicen ingeniería reversa para la capa de dominio para listar. Capturen los resultados de las pruebas de unidad.
5. Propongan y ejecuten una prueba de aceptación. Capturen las pantallas correspondientes.

1. a) Vemos que al añadirlo, no hay ningún mensaje de confirmación o error. Y al listar en Listar, no se visualiza la nueva información.

b) Se comprueba porque Listar sigue igual aún llenando Adicionar con la información nueva

Aún dando en Adicionar no se añade la nueva información:



c) No, debería dar algún mensaje de confirmación, alguna pista sobre cómo llenar el formulario o si hay alguna excepción o dato inválido antes de añadirlo, Y por supuesto debería actualizarse Listar.

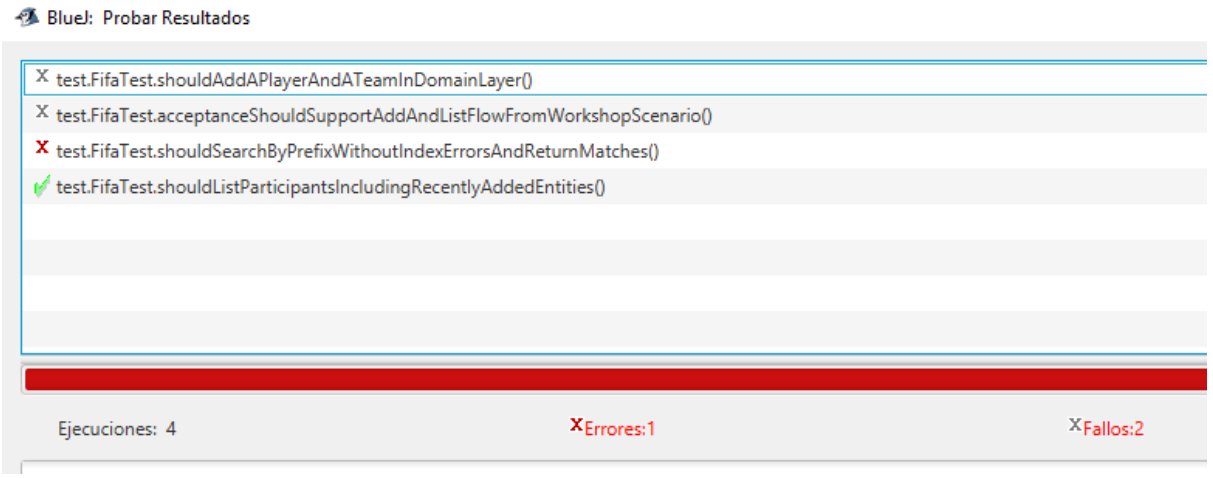
d) El método responsable es prepareAreaAdd()

```
/**
 * Prepare adiction area
 * @return adiction area
 */
private JPanel prepareAreaAdd(){

    JPanel fields = new JPanel(new GridLayout(13,1));
    fields.add(new JLabel("Nombre"));
    fields.add(name);
    fields.add(new JLabel("Posicion"));
    fields.add(position);
    fields.add(new JLabel("Minutos jugados"));
    fields.add(minutes);
    fields.add(new JLabel("Director (para equipo) o Club (para jugador)"));
    fields.add(managerClub);
    fields.add(new JLabel("Uniforme (solo para equipos)"));
    fields.add(uniform);
    fields.add(new JLabel("Valor de mercado (solo para jugadores)"));
    fields.add(value);
    fields.add(new JLabel("Jugadores (solo para equipos)"));
```

e) En la capa de domain. El responsable es addTeam() y addPlayer() en Fifa.

```java
package test;

import domain.Fifa;
import org.junit.Test;

import static org.junit.Assert.*;

public class FifaTest {

    @Test
    public void shouldAddAPlayerAndATeamInDomainLayer(){
        // Arrange
        Fifa fifa = new Fifa();

        // Act
        fifa.addPlayer("MESSI", "1420", "D", "15000000", "Inter");
        fifa.addTeam("ARGENTINA", "1620", "J", "Scaloni", "Azul-blanco", "MESSI");

        // Assert
        assertEquals(7, fifa.numberParticipants());
    }

    @Test
    public void shouldListParticipantsIncludingRecently AddedEntities(){
        // Arrange
        Fifa fifa = new Fifa();
        fifa.addPlayer("MESSI", "1420", "D", "15000000", "Inter");
```

2. Observamos que hay errores y fallos.



3. Es el método prepareAreaList() en FifaGUI.Y en domain es el método toString() de Fifa.

4.
```java
private void  actionAdd(){
    if (players.getText().trim().equals("")){
        fifa.addPlayer(name.getText(),minutes.getText(),position.getText(),value.getText(),managerClub.getText());
    }else{
        fifa.addTeam(name.getText(),minutes.getText(), position.getText(), managerClub.getText(),uniform.getText(),players.getText());
    }
}
```

El problema estaba en el orden de los parámetros en actionAdd de FifaGUI

Vemos que ahora si se ejecutan correctamente

test.FifaTest.shouldSearchByPrefixWithoutIndexErrorsAndReturnMatches()

test.FifaTest.shouldListParticipantsIncludingRecentlyAddedEntities()

5.
```java
@Test
public void AcceptanceTestAddList(){
    // Arrange
    Fifa fifa = new Fifa();

    // Act
    try {
        fifa.addPlayer("MESSI", "1420", "D", "15000000", "Inter");
        fifa.addTeam("ARGENTINA", "1620", "J", "Scaloni", "Azul-blanco", "MESSI");
    } catch (FifaException e) {
        fail("Threw an exception");
    }
    String listing = fifa.toString();

    // Assert
    assertTrue(listing.contains(">MESSI"));
    assertTrue(listing.contains(">ARGENTINA"));
}
```

C Fortaleciendo. Adicionar robusta. [En lab04.doc fifa2026.astah *.java]

(NO OLVIDE BDD – MDD)

El objetivo es fortalecer la funcionalidad de adicionar para hacerla más robusta. Para cada uno de los siguientes casos realice los pasos del 1 al 4.

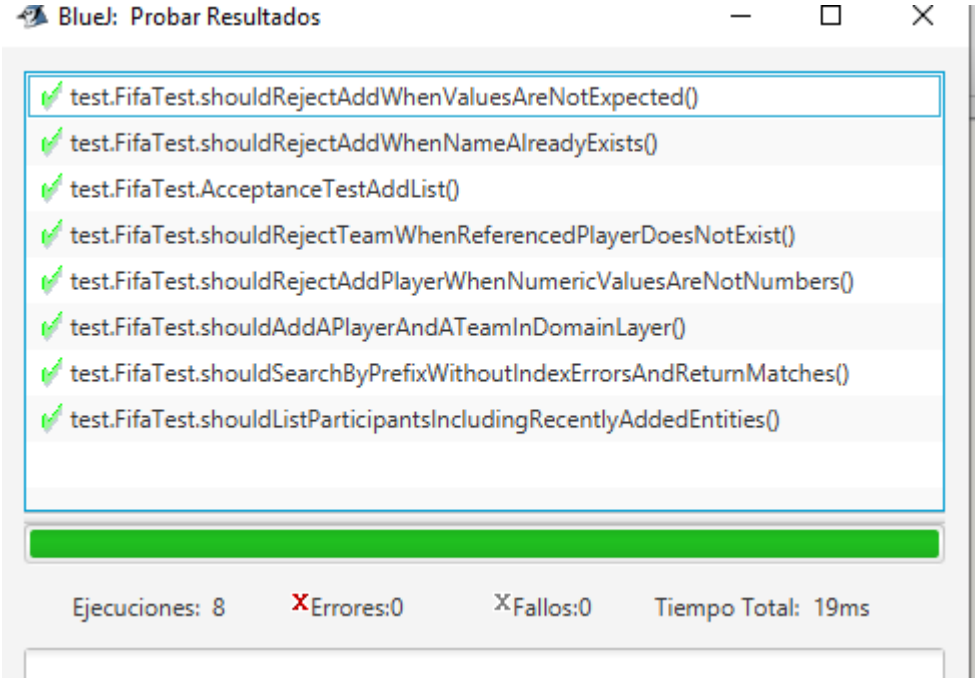
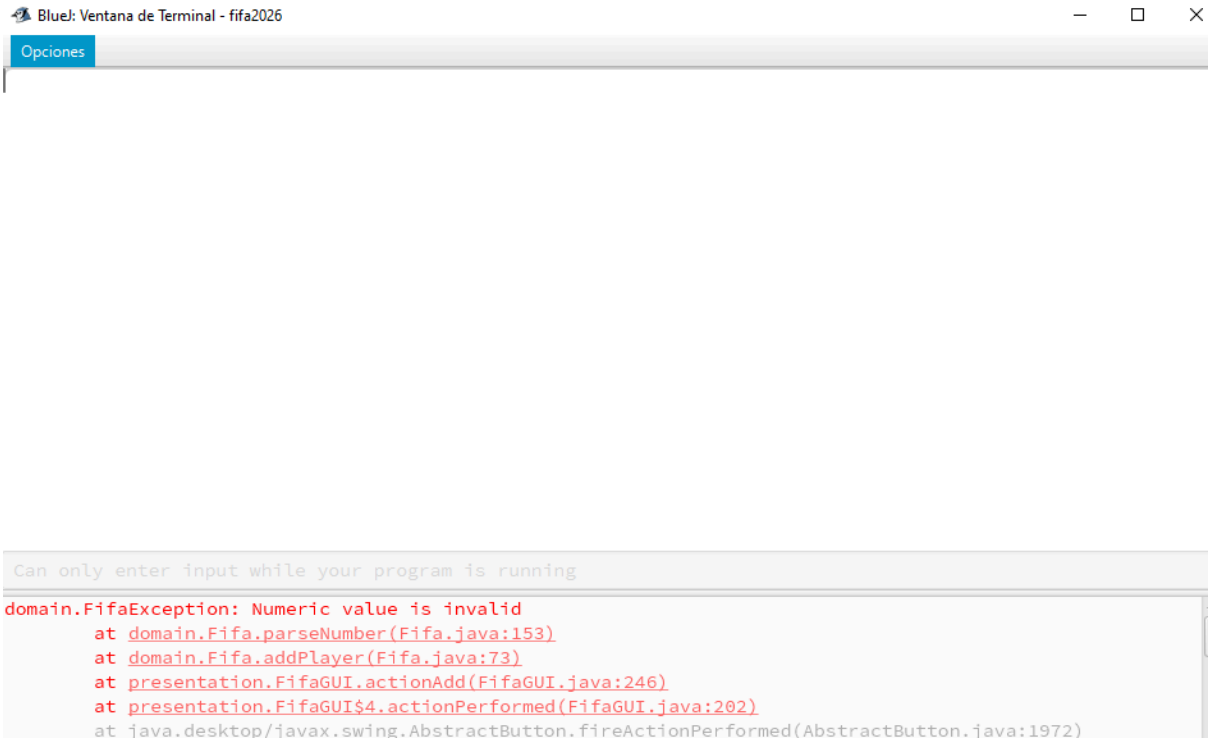
(i) ¿Y si el nombre ya existe?
(ii) ¿Y si los valores numéricos no son numéricos?
(iii) ¿Y si los valores no son los esperados?
(iv) Proponga una nueva condición

1. Propongan una prueba de aceptación que genere el fallo. (a) ¿Se generó alguna excepción? (b) ¿Por qué?
2. Analicen el diseño dinámico (diagrama de secuencia). Para hacer la funcionalidad robusta: (a) ¿Qué método debería lanzar la excepción? (b) ¿Qué métodos deberían propagarla? (c) ¿Qué método debería atenderla? Explique claramente.
3. Construya la solución propuesta. Capture los resultados de las pruebas de unidad.
4. Ejecuten nuevamente la aplicación con el caso de aceptación propuesto en 1. (a) ¿Qué sucede ahora? Capturen la pantalla.

```java
@Test
public void shouldSearchByPrefixWithoutIndexErrorsAndReturnMatches(){
    // Arrange
    Fifa fifa = new Fifa();
    try {
        fifa.addPlayer("MESSI", "1420", "D", "15000000", "Inter");
        fifa.addTeam("ARGENTINA", "1620", "J", "Scaloni", "Azul-blanco", "MESSI");
    } catch (FifaException e) {
        fail("Threw an exception");
    }

    // Act
    String result = fifa.search("AR");

    // Assert
    assertTrue(result.contains(">ARGENTINA"));
```

1. a) No se generó ninguna
   b) Porque no está definida para esos casos específicos

2. a) Deberia propagarla addPlayer() y addTeam() pues son los métodos principales, además de los métodos auxiliares. Deberia atenderla addSome()



3.
4. Si funciona pero con excepciones